

软件项目开发综合实践 课程报告

《个人学习任务管理与数据分析系统》

姓名	学号	项目标识	完成日期
佟政慷	239001013	SPD-1013	2026-09-04

一、项目概述

本项目是「软件项目开发综合实践」的课程作业，题目为《个人学习任务管理与数据分析系统》，项目标识 SPD-1013（取自学号 239001013 的后四位）。它解决的是我自己的真实问题：作业、实验、考试的信息分散在微信群、教材扉页和便利贴上，临近截止才发现漏了一项。系统把这些任务统一录入，按课程和截止日期集中管理，再把任务数据汇总成图表，用来回答「哪门课欠得最多」「这周还剩几件事」这类问题。

交付形态是本地 Web 应用：命令行执行 `python app.py` 启动后，浏览器访问 `127.0.0.1:5000`，登录即用，不依赖任何外部服务或云端账号。自写代码合计约 4970 行——应用 2083 行 Python、测试 831 行 Python、模板 1033 行 HTML、样式与脚本 1025 行；Bootstrap 与 Chart.js 使用本地副本，断网也能完整运行。

1.1 技术选型

层次	选型	选择理由
后端框架	Flask 3.1.1	只需要「路由 + 模板」两件事，蓝图分模块足够；不引入 ORM，SQL 全部手写，每一次查询都讲得清
数据库	SQLite 3（标准库 <code>sqlite3</code> ）	单文件、零配置、随项目拷走；单用户场景下并发不是瓶颈
前端	Bootstrap 5.3.3	响应式栅格与组件齐全，不必自己写一套 CSS 框架
图表	Chart.js 4.4.7	环形图、柱状图、条形图、折线图一次搞定，本地引入即可
测试	pytest 8.3.4	与 Flask 的 <code>test_client</code> 配合好，夹具机制适合「每个用例一个干净库」

二、功能清单

2.1 基础功能

模块	实现要点
任务管理	新增 / 编辑 / 删除 / 完成状态一键切换；字段含名称、所属课程、截止日期、优先级（高·中·低）、完成状态、备注
查询与排序	关键词搜索同时匹配名称与备注；课程、状态、优先级、截止日期区间四类条件任意组合（AND）；五个字段可升降序；每页 10 条分页
数据统计	总任务数、已完成、未完成、完成率四项指标；任务列表页与仪表盘共用同一套统计口径
数据持久化	SQLite 单文件存储，四张表（ <code>users</code> / <code>courses</code> / <code>tasks</code> / <code>operation_logs</code> ），任务与课程之间为外键关联，重启后数据保留

2.2 扩展功能

扩展方向	实现内容
数据可视化	四张图表：完成情况环形图、各课程任务堆叠柱状图、优先级分布条形图、近 7 天完成趋势折线图；「刷新数据」按钮走 REST 接口原地更新，不刷新页面
截止提醒	派生五种到期状态（已完成 / 已逾期 / 今天到期 / 3 天内 / 正常），列表行左侧色条区分；导航栏铃铛角标 + 三档预设视图
日历视图	月历网格按截止日期铺任务，点某天的「+」直接以该日期新建；整月一次查库后分组，避免 42 个格子查 42 次
导入导出与备份	CSV（带 BOM，Excel 双击不乱码）与 Excel 双格式导出，导出结果跟随当前筛选；CSV/Excel 导入支持部分成功；一键数据库快照
REST API	/api 下 10 个接口，读接口允许已登录会话，写接口强制 X-API-KEY 请求头
智能分类	任务名称输入框失焦后按关键词规则推荐所属课程与优先级，并给出命中依据，可一键采纳
操作日志	增删改、导入导出、备份、改密码全部留痕，数据管理页可追溯

三、系统设计

3.1 分层结构

```
views/      路由与参数解析，这一层不写任何 SQL
  ↓ 只调用 service
services/   业务规则：字段校验、派生到期状态、统计口径、导入导出、智能分类、鉴权
  ↓ 只调用 repo
models/     唯一写 SQL 的一层，条件值全部用 ? 占位符传参
  ↓
db.py       g 作用域连接，每次连接都开 PRAGMA foreign_keys = ON
```

分层带来两处实际收益。一是页面和 REST 接口共用同一套 service，/tasks/ 与 /api/tasks 的筛选、排序参数完全一致，不会出现「页面上 7 条、接口返回 5 条」；测试里专门有一条用例断言首页 HTML 里的四个统计数字与 /api/stats 返回值逐项相等。二是 SQL 集中在 models/ 一层，排查注入风险时只需要看这几个文件。

3.2 数据库设计

表	关键字段	设计说明
users	username / password_hash / display_name / student_id	只存 PBKDF2 哈希，不存明文；姓名与学号入库，页面信息条从数据库读，不写死在模板里
courses	name UNIQUE / teacher / color	name 加唯一约束，避免同一门课录成两条把统计拆开；color 供图表与日历统一配色
tasks	title / course_id / due_date / priority / status / note / completed_at	course_id 外键指向 courses.id，ON DELETE SET NULL——删课程不连带删任务，任务退化为「未分类」；priority 与 status 用 CHECK 约束把非法值挡在库外；completed_at 是趋势图唯一数据来源
operation_logs	action / target / detail / created_at	target 存任务标题快照，任务被删除后日志依然可读

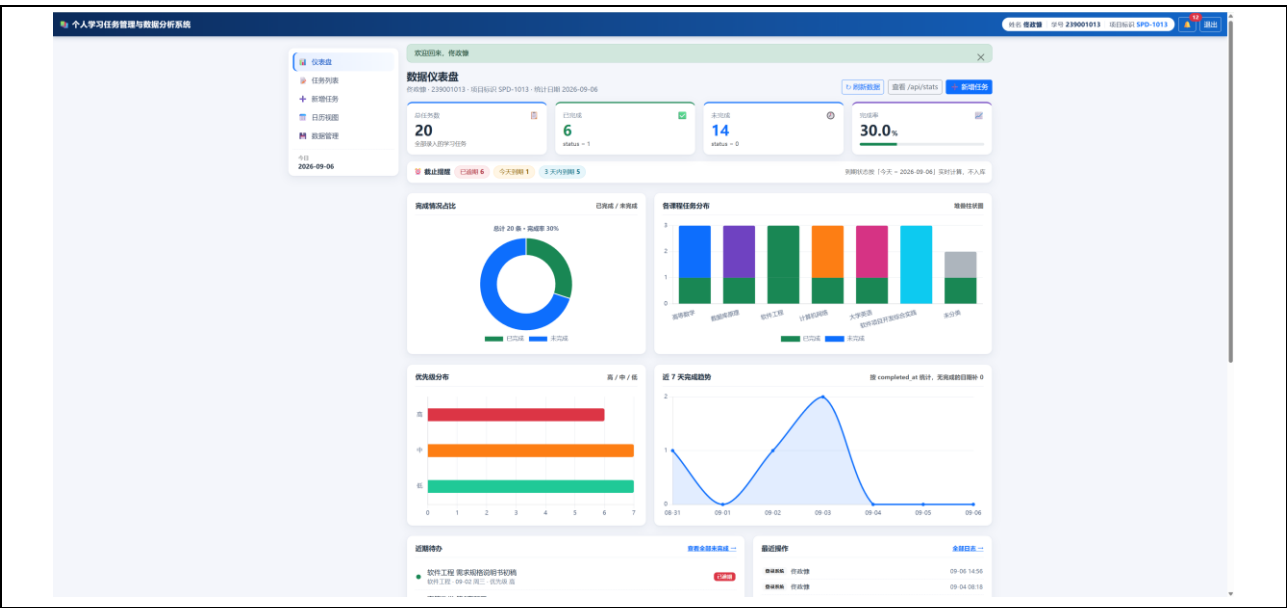
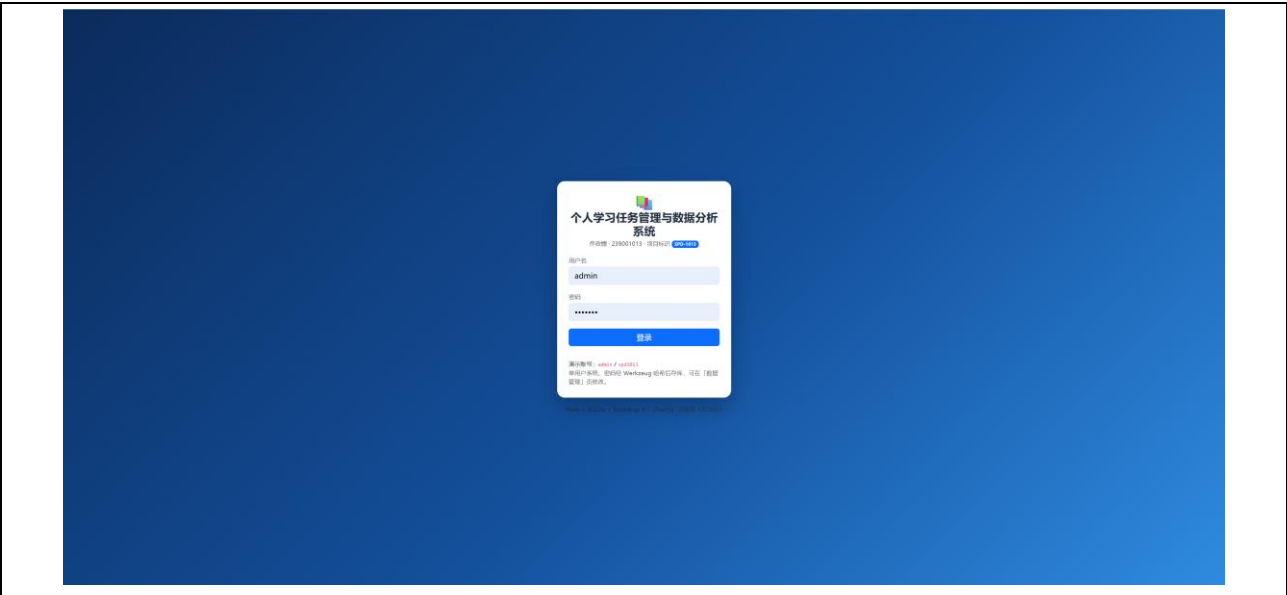
索引按实际查询习惯建了三个：due_date（日历与默认排序）、status（统计）、course_id（按课程筛选）。

3.3 三个关键取舍

- 优先级存整数 1/2/3，而不是「高/中/低」文字：数据库里 ORDER BY priority 直接就是高→中→低；若存中文，SQLite 默认按 UTF-8 字节比较，「中(U+4E2D) < 低(U+4F4E) < 高(U+9AD8)」，升序会排成中→低→高，毫无意义。中文标签只在显示层由模板过滤器转换。
- 截止日期存 YYYY-MM-DD 文本：SQLite 没有真正的日期类型，而 ISO 格式文本的字典序恰好等于时间序，所以 `due_date >= ? AND due_date <= ?` 这样的区间筛选可以直接比较字符串，不需要任何转换函数。
- 到期状态不入库、每次查询现算：如果把「已逾期」写进表，就必须有定时任务每天刷新，而任何一次漏跑都会让页面显示错误状态。改为按「今天」派生后永远准确，代价只是一次比较。

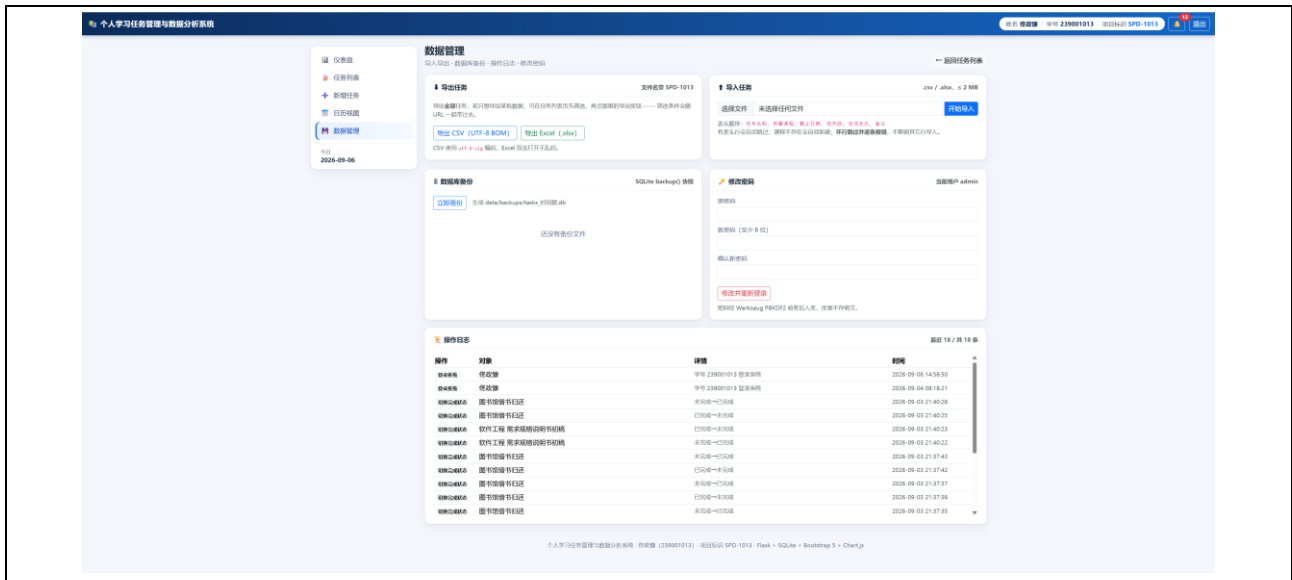
四、运行效果

以下截图均在本机浏览器 127.0.0.1:5000 与命令行下采集。页面顶部信息条由 `base.html` 统一渲染，因此每张页面截图都自带姓名、学号与项目标识 SPD-1013。



[illegible]

佟政慷 239001013 SPD-1013 第 4 页



五、核心代码解析

5.1 排序字段白名单——ORDER BY 无法参数化时如何防注入

摘自 spdapp/models/task_repo.py (26 行, 省略了中间的 SORT_LABELS 定义)

```
# SQL 的 ORDER BY 不支持 ? 占位符, 只能字符串拼接,
# 所以排序字段必须走白名单, 杜绝 URL 参数直接进 SQL。
ALLOWED_SORT = {
    "due_date": "t.due_date",
    "priority": "t.priority",
    "created_at": "t.created_at",
    "title": "t.title",
    "status": "t.status",
}
DEFAULT_SORT = "due_date"

def build_order(sort, order):
    """生成 ORDER BY 子句; 非法字段一律回落到默认排序。"""
    column = ALLOWED_SORT.get(sort or DEFAULT_SORT, ALLOWED_SORT[DEFAULT_SORT])
    direction = "DESC" if str(order).lower() == "desc" else "ASC"
    # 追加次级排序键, 保证同值行的顺序稳定, 翻页时不会出现重复或漏项
    return f" ORDER BY {column} {direction}, t.priority ASC, t.id DESC"

def list_tasks(filters=None, sort=None, order=None, limit=None, offset=0):
    """按筛选条件 + 排序取任务列表, limit 为 None 时取全部 (导出用)。"""
    where, params = build_where(filters or {})
    sql = _SELECT + where + build_order(sort, order)
    if limit is not None:
        sql += " LIMIT ? OFFSET ?"
        params += [int(limit), int(offset)]
    return get_db().execute(sql, params).fetchall()
```

项目里所有用户输入都以 ? 占位符传参 (build_where 生成的条件值也一样), 唯独 ORDER BY 的字段名不能用占位符——SQL 语法不允许, 只能拼字符串, 这正是注入最容易被放过的地方。做法是让排序字段先过 ALLOWED_SORT 映射: URL 里传什么, 落进 SQL 的都只可能是这五个受控列名之一, 非法值直接回落默认排序。测试里真的传了 sort=id; DROP TABLE tasks——, 结果页面正常返回、20 条数据完好、表还在。

5.2 完成趋势补零——SQL 不会为「零完成」的日期产出行

摘自 spdapp/services/stats_service.py (15 行)

```
def get_trend(days=None, today=None):
    """近 N 天完成趋势。

    SQL 的 GROUP BY 不会为「当天零完成」生成结果行，直接拿去画折线会缺点、
    日期还会错位，所以这里按完整日期序列补零。
    """
    days = days or current_app.config["TREND_DAYS"]
    today = today or today_str()
    start = shift(today, -(days - 1))
    hit = {r["d"]: r["cnt"] for r in stats_repo.completed_per_day(start)}
    return [
        {"date": shift(start, i), "label": shift(start, i)[5:],
         "count": hit.get(shift(start, i), 0)}
        for i in range(days)
    ]
```

底层查询是 `SELECT date(completed_at) AS d, COUNT(*) AS cnt ... GROUP BY d`，它只返回有完成记录的日期。如果把结果直接交给折线图，没完成任务的那天不是「0」而是整点缺失，日期轴会被压缩错位——09-01 的点会画到 09-03 的位置上。所以先在 Python 里生成完整的 7 天日期序列，再用查询结果去填，缺的补 0。测试断言了三件事：长度恒等于 `TREND_DAYS`、日期严格升序、首日为 0 而末日为 1。

5.3 派生到期状态——不入库，按「今天」现算

摘自 `spdapp/services/task_service.py` (19 行)

```
def derive_state(task, today=None, remind_days=None):
    """派生到期状态。

    刻意不入库：存进数据库就需要定时任务刷新，且一定会和真实日期不一致；
    改为查询时按「今天」现算，永远准确。
    """
    if task["status"] == 1:
        return "done"
    today = today or today_str()
    if remind_days is None:
        remind_days = current_app.config["REMIND_DAYS"]
    due = task["due_date"]
    if due < today:
        return "overdue"
    if due == today:
        return "today"
    if due <= shift(today, remind_days):
        return "soon"
    return "normal"
```

五种状态按「已完成 → 已逾期 → 今天到期 → N 天内 → 正常」的顺序短路判断，顺序本身就是优先级：一个已完成的任务即使日期过了也不该显示成逾期，所以 `status` 的判断必须排在最前面。三处日期比较全是字符串比较，靠的就是 3.3 里 ISO 文本字典序等于时间序这一条；`shift()` 只做日期加减，不涉及时区。提醒天数从配置读取，改成 5 天提醒不用动任何 SQL 或模板。这个函数同时服务列表行色条、日历色块和 REST 接口的 `state` 字段，三处口径必然一致；测试用 -9 / -1 / 0 / +3 / +4 天五个日期把五个分支全部走了一遍。

六、测试与问题解决

6.1 测试方案

用 `pytest` 配合 Flask 的 `test_client` 做接口级测试：每个用例通过夹具拿到一个全新的临时数据库文件，跑完即删，用例之间互不影响。刻意没用 `:memory:`——SQLite 的内存库是「一个连接一个库」，而应用每个请求都新建连接，用内存库会出现「上一步写进去、下一个请求查不到」的假失败。

测试文件	覆盖内容	用例数
<code>test_task_crud.py</code>	登录保护、增删改查、字段校验、完成时刻记录、操作日志、跳转地址安	14

	全	
test_filter_sort.py	关键词、四类条件组合、预设视图、五字段排序、分页边界、非法参数	14
test_io.py	CSV/Excel 导出、部分成功导入、编码兼容、数据库备份、修改密码	13
test_stats.py	统计口径、零除保护、趋势补零、派生状态矩阵、页面与接口一致性	11
test_api.py	接口鉴权、CRUD 全链路、校验错误码、智能分类	10
合计	python -m pytest -q → 62 passed (约 18 秒)	62

```
PS D:\SoftWare\Project\软件> python -m pytest -q
..... [100%]
62 passed in 17.82s
PS D:\SoftWare\Project\软件>
```

6.2 专门覆盖的边界与安全场景

场景	期望行为	结果
排序字段传 id; DROP TABLE tasks--	白名单丢弃，回落默认排序，表与数据完好	通过
手改 URL 传 status=abc、due_from=2026-13-45	非法值忽略并返回全部数据，而不是 500	通过
空数据库访问首页	完成率显示 0.0% (0/0 不短路就是 ZeroDivisionError)	通过
页码传 page=99	夹到最后一页，不出现空白页	通过
删除后跳转参数传 //evil.example.com/	只接受站内地址，否则回落任务列表	通过
带登录会话但不带 X-API-KEY 调写接口	401，会话本身不足以授权接口写操作	通过
导入含 3 条坏行的 CSV	好行入库、坏行逐条报出原因，不整体回滚	通过

6.3 真实调试经历：Chart.js「Canvas is already in use」

现象：仪表盘做好后加了「刷新数据」按钮。第一次点击一切正常，第二次点击四张图表全部消失，控制台报错 Canvas is already in use - Chart with ID '0' must be destroyed before the canvas with ID '...' can be reused。

定位：先怀疑数据，用 F12 网络面板确认 /api/stats 每次都返回了完整 JSON，说明接口没问题；再回读报错文本里的 must be destroyed / can be reused，这两个词指向的是对象生命周期，不是数据。查代码发现我把「初始化」和「刷新」写成了同一个函数——刷新时对同一个 canvas 又执行了一次 new Chart()。Chart.js 4 按 canvas 注册实例，同一张画布不允许挂两个实例，第二次构造直接抛错；而抛错发生在画布被清空之后，所以视觉上表现为「图表消失」。

修复：把四个图表实例存进一个 charts 对象，初始化只做一次；刷新只改 data 再 update()。

```

/* 刷新只改 data 再 update(), 不对同一个 canvas 重新 new Chart (见文件末尾说明) */
function applyUpdate(data) {
  if (charts.status) {
    charts.status.data.datasets[0].data = [data.summary.done, data.summary.undone];
    charts.status.options.plugins.title.text = statusTitle(data.summary);
    charts.status.update();
    markEmpty('chartStatus', !data.summary.total);
  }
  ... // 其余三张图同理
}

```

验证：连点「刷新数据」十余次，控制台无报错，四张图表数字随库内数据同步变化，Chart.js 的过渡动画也保留了下来（如果确实需要重建，比如切换图表类型，则必须先 `chart.destroy()` 再 `new`）。

收获：报错信息里的动词往往直接指向问题类别——出现 `destroyed` / `reused` 就该先查生命周期，比反复怀疑数据快得多。这段结论以注释形式留在了 `dashboard.js` 末尾，防止以后自己再犯。

七、个性化设计

- 身份信息内建：姓名、学号、项目标识写在 `config.py`，由 `base.html` 一处渲染成顶部信息条，因此任意页面的截图都自带个性化信息，不需要事后补拍；PROJECT_CODE 由「SPD- + 学号后四位」推导，换学号会自动跟着变。
- 演示数据贴合真实课表：6 门课程（含「软件项目开发综合实践」）与 20 条任务，覆盖已逾期、今天到期、3 天内、已完成各种状态，第一条即为「软件项目开发综合实践-SPD1013 系统开发」。
- 导出文件名带项目标识与时间戳，如 `SPD-1013_任务清单_20260904_2130.csv`，多次导出不会互相覆盖，也便于在附件里认出是谁的作业。
- 智能分类用可解释的规则而不是模型：课程别名表加优先级关键词表，「实验 / 报告 / 答辩 / 考试」判高，「预习 / 复习 / 整理」判低，并把命中的依据原样返回给前端显示，用户知道为什么给出这个建议，也能一键忽略。

八、总结与不足

完成度：任务书要求的全部基础功能与四组扩展功能均已实现，62 条自动化测试全部通过，并在本机做过一轮端到端手动验证（登录、五个页面、筛选排序、导入导出、备份、接口调用）。整个过程中收获最大的不是把功能堆齐，而是几个「想清楚再写」的地方：优先级为什么存整数、日期为什么存文本、到期状态为什么不入库——这三个决定后来分别省掉了排序逻辑、日期转换函数和一个定时任务。

不足与后续方向：

- 单用户设计。`users` 表虽然建好了，但没有注册流程与多用户数据隔离，`tasks` 表也没有 `user_id` 列，要支持多人需要在表结构与每一条查询上补条件。
- API_KEY 明文写在配置里、不过期、无速率限制，只够本机演示，不是生产级鉴权；真要上线应换成带过期时间的令牌并启用 HTTPS。
- 统计只做到「已完成 / 未完成」的计数与趋势，还没有「预计耗时 vs 实际耗时」这类分析——那需要给 `tasks` 表加两个字段并在完成时记录，属于下一步。
- 前端没有做打包压缩，样式是手写 CSS 变量；页面再多下去应该考虑组件化。

九、AI 使用说明

本项目在 AI 辅助下完成。选题、功能范围、技术选型（Flask + SQLite + Bootstrap + Chart.js）、数据表设计与演示数据由我确定；AI 用于按我的方案生成代码框架、页面模板与部分测试用例，并在我

追问时解释实现取舍。所有代码我逐文件读过，在本机运行、自测并改过，报告文字由我撰写，文中的设计理由与调试过程我都能逐条复述。